Handoff Documents Summary -- 2026-10-29

Overview

Two handoff documents exist for this project:

1. **`opencode-browser-handoff-er.txt`** (repo root) -- Comprehensive emergency handoff for QuickJS network browser work (~550 lines). Written by a prior opencode session starting 2026-09-19. Covers YouTube rendering through a custom QuickJS DOM engine.
2. **`xo/handoff_notes.md`** (xo directory) -- Shorter handoff for tomom dashboard & relay harness testing (35 lines, 2026-10-25).

Document 1: Network Browser Handoff (opencode-browser-handoff-er.txt)

Goal

Drive real web pages (YouTube as target) through a QuickJS-based network browser to achieve functional parity with Chromium: real pages render + are interactive through the manager, zero JS errors, end-to-end.

Key Components

- `network_browser_manager.c` -- Manager/orchestrator with resident worker, curl, cookies
- `ops/nb_js_worker.c` -- QuickJS worker: DOM engine, page scripts, event loop, render-back
- `nb_dom.c`, `nb_css.c` -- DOM and CSS implementations
- Vendored QuickJS in `../js/`

Acceptance Criteria

- Opening youtube.com yields static render with no JS errors
- `#search-icon-legacy` wired and live
- Native search click/enter drives page navigation
- Results URL lands in page.state
- Cookies persist
- `make check` stays 60 PASS / 0 FAIL

What Works (as of 2026-09-19)

- Resident worker keepalive (LIVE|drain frames every ~1800ms)
- Manager LOAD/EVAL quiet budget (WORKER_LOAD_QUIET_MS=90000)
- `performance` host API (killed YouTube's ReferenceError)
- Address-bar search fallback (`go:<query>` -> `/results?search_query=<enc>`)
- End-to-end verified: go:youtube -> ready -> search value set -> nav -> results URL in page.state
- `make check` = 60 PASS / 0 FAIL

Key Fixes Shipped (2026-09-20 through 2026-09-21)

1. **Buffer parsing** -- Fixed strtok corrupting HTTP body extraction
2. **customElements engine** -- Real `customElements.define/get/whenDefined/upgrade` with registry, shadow roots, template content
3. **nodeType fix** -- Added nodeType=1/3/9/11 to DOM wrappers (root cause of kevlar `Error("0c")`)
4. **KeyboardEvent/InputEvent subclasses** -- Were empty stubs, now have proper init-key allowlists
5. **Element focus/blur** + `activeElement` support
6. **Search nav** -- Engine proxies Enter on `input[name=search_query]` to navigate to `/results?search_query=`

Protocol & File Contract

- Worker daemon protocol: frame-based (`XXXXX\n` length + content)
- Commands: `LOAD\n<body>`, `EVAL\n<js>`, `QUIT`

- Replies: `STATUS`, `LIVE|drain|%u`, `RENDER`, `WERR`, `=>BOX_SET`, `=>NAV_REQUESTED`
- Houses: `/tmp/nbhouseNN` with ephemeral artifacts (crash wipes /tmp)
- Manager request: `#.desktop/network_browser_request.txt` with `go:`, `eval:`, `go:eval:` commands
- Status: `network_browser_status.state.txt`; Page state: `page.state` with rows `TITLE|`, `URL|`, `TEXT|`, `LINK|`, `BOX|`

Next Steps (Rung-Based Plan)

- **RUNG 0** (DONE): Home loads error-free; masthead input materializes; typed Enter navigates
- **RUNG 1** (NEXT): Run kevlar's real masthead materialization (yt-searchbox custom element upgrade so `<form>`, `#search-icon-legacy` appear)
- **RUNG 2**: Wire materialized search UI natively (click + Enter through kevlar's own bind)
- **RUNG 3**: Drive same flow through kevlar's FX branches AND chromium path; diff URL + tree renders
- **RUNG 4**: Promote verified interaction into regression harness (make check 60->61)

Build Commands

- Worker: `gcc -std=gnu11 -O2 -D_GNU_SOURCE "-DCONFIG_VERSION=\"2026-06-04\"" -fwrapv -pthread -I"../js" -o ops/+x/nb_js_worker.+x ops/nb_js_worker.c nb_dom.c nb_css.c ../js/quickjs.c ...`
- Manager: `gcc -std=c11 -Wall -O2 -o "+x/network_browser_manager.+x" network_browser_manager.c nb_dom.c`

Gotchas

- Python `select() + buffered read(1)` harnesses LIE about frame delivery -- redirect to files
- Page.js stack line numbers are offsets into kevlar bundle (~43K lines), not the actual page.js
- Quoted-path rule: house paths contain &, #, !, spaces -- always quote
- Crash/reboot wipes /tmp houses -- artifacts are ephemeral

Document 2: tomom Dashboard & Relay Handoff (xo/handoff_notes.md)

Goal

Test the tomom chatbot (C-based neural network with Mixture of Experts) through relay interface chat harnesses.

What Was Done

- **Fixed `http_server.c` body parsing bug** -- `strtok()` was corrupting buffer before body extraction
- **Tested relay harnesses:**
- iqa-fork `ask.sh` -- working LLM relay to LAN Mac ollama (10.0.0.144:11434, gemma3:270m)
- HTTP relay -- now works after fix
- Direct binary -- works, weak responses (untrained model)
- Local ollama (llama3:latest) -- works, slow startup (~10s)
- **Assessed harness options:**
- gem-dev -- concurrent TPMOS agent framework, not an LLM relay
- iqa-fork -- valid LLM relay
- pal-chat-irc -- key-injection testing framework, not LLM relay
- bot5 -- concurrent agent framework, not LLM relay
- **No "middle-house" solution exists** -- zero results in codebase
- **Created report:** `xo/relay_harness_test_report.md` + `.html`

Remaining Issues

- button.sh SCRIPT_DIR resolution fails due to emoji in directory name -> `khtpm_core_render.+x` not found at runtime

Next Steps

1. Fix button.sh path resolution (emoji directory issue)

2. Test full `button.sh run` in X11 terminal (needs raw mode)
3. Wire up khtpm interactive stack (keyboard_input, prisc+x) for .chtpm interactivity

Files Changed

- `http_server.c` -- buffer parsing fix
- `button.sh` -- partial path fix (RENDERER path)
- `xo/relay_harness_test_report.md` -- new
- `xo/relay_harness_test_report.html` -- new
- `xo/handoff_notes.md` -- new

Cross-Reference

Both handoffs are complementary: - The **network browser handoff** is the main codebase work -- building a QuickJS browser engine - The **tomom handoff** is auxiliary work -- testing an LLM chatbot relay harness

Related handoff documents: - `2026-10-18-BROWSER-LANE-HANDOFF.md` (HQ-IQ-BOOK/08-roadmap) -- browser lane KPIs and build commands - `from-grok-game-dev-office.md` (HQ-IQ-BOOK/13.agent-coms) -- game-dev office handoff - `HANDOFF-2026-10-13.md` -- daily handoff for 2026-10-13